DIGITAL SYSTEM DESIGN WITH VERILOG USING AI TOOLS

Chapter 11

SoC Programming

Peripheral, Address Map, C, and Full System

C Program — RISC-V Instructions — Address Bus — Peripheral — FPGA Pin

RISC-V RV32I SoC from Chapter 10 · DE0 FPGA | VS Code + AI Assistant · RISC-V GCC · Icarus Verilog · Quartus II

01

From CPU to SoC

Why this matters

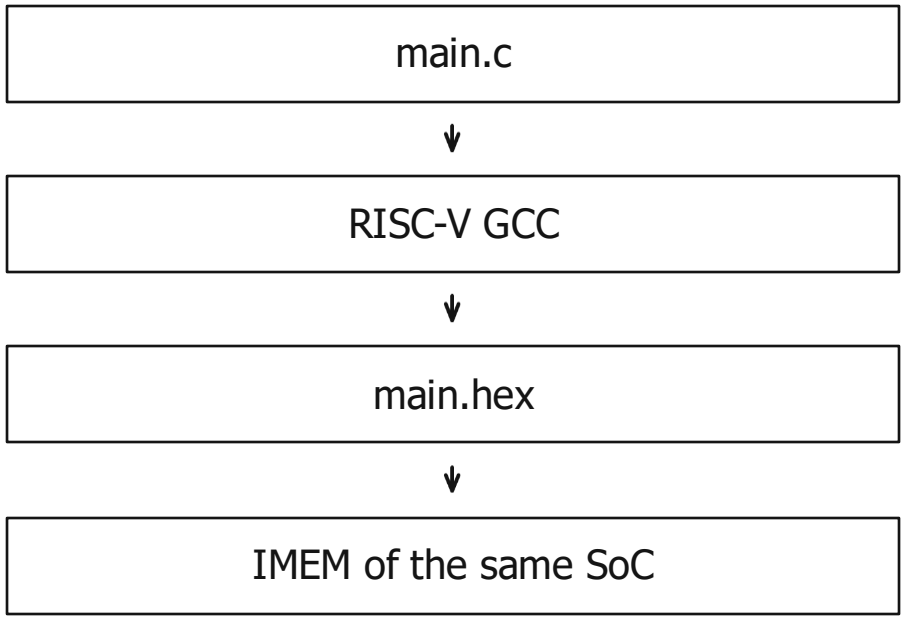
The Chapter 10 hardware stays the same. What changes is how we program it: a C statement must become instructions the CPU supports and bus transactions the hardware understands.

Where We Are Now

In Chapter 10, we built:

- rv32i_core
- Instruction Memory — IMEM
- Data Memory — DMEM
- Address Decoder
- GPIO
- LED / Switch / KEY / 7-segment interface

New in Chapter 11: the C toolchain



Chapter 11 does not replace the CPU.

We program the same hardware using C.

What Changes from Chapter 10?

	Chapter 10	Chapter 11
Program language	RISC-V Assembly	C
Main software tool	RARS	RISC-V GCC
Program image	gpio.hex	main.hex
CPU	rv32i_core	Same
IMEM / DMEM	Existing	Same
GPIO / Bus	Existing	Same
Main new concept	Build CPU/SoC	Hardware–Software Interface

Hardware does not know whether machine code came from Assembly or C.

The Central Question

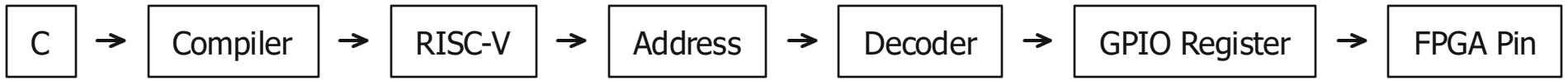
Consider:

```
LED = 0x3FF;
```

This looks like a normal C assignment.
But there is no variable called "LED" inside the CPU.

So how does this statement turn on a physical LED?

By the end of this chapter, we will follow the complete path:



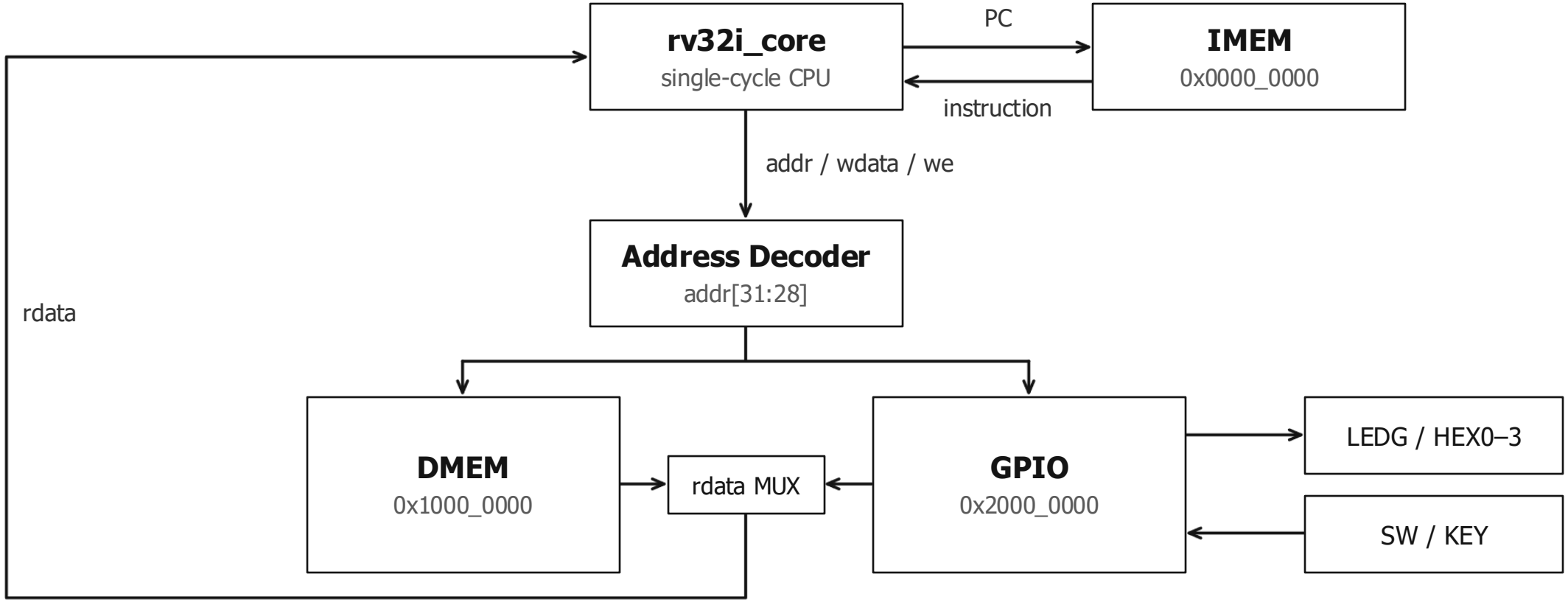
System Contract for This Chapter

We reuse the Chapter 10 system.

Item	Configuration
CPU	rv32i_core, single-cycle
IMEM	0x0000_0000, 256 words = 1 KB
DMEM	0x1000_0000, 256 words = 1 KB
GPIO	0x2000_0000
LED	GPIO +0x00, write
SW	GPIO +0x04, read
KEY	GPIO +0x08, read
HEX	GPIO +0x0C, write, 16-bit value
IMEM data access	Not available

Important: the software must fit the hardware that actually exists.

Review: The Existing SoC



SoC = CPU + Memory + Interconnect + Peripherals

One Instruction Can Reach the Outside World

Assembly:

```
sw t0, 0(s0)
```

s0 = 0x1000_0000 → writes RAM (DMEM)

s0 = 0x2000_0000 → writes the GPIO LED register → LEDG

If s0 = 0x2000_0000, the store is not writing RAM. It is writing the GPIO LED register.

Same CPU instruction — different hardware destination.

The destination is selected by the **address**.

Memory-Mapped I/O

A peripheral is represented as one or more addresses.

The CPU already knows two important operations:

```
lw    # read from an address
sw    # write to an address
```

Therefore:

LW/SW + Address Map
= I/O Access

No special instruction such as **TURN_LED_ON** is required.

02

Memory Map and Address Decoder

Why this matters

The address selects the device. The memory map is the contract that hardware and software must both follow.

The Memory Map Is a Contract

A **memory map** defines who owns each address range.

0x0000_0000 – 0x0000_03FF	**IMEM — 1 KB**
unmapped	no device
0x1000_0000 – 0x1000_03FF	**DMEM — 1 KB**
unmapped	no device
0x2000_0000 – 0x2000_000F	**GPIO registers**

One address must have one documented meaning.

System Memory Map

Region	Base Address	Implemented Size	Function
IMEM	0x0000_0000	1 KB	Instruction ROM
DMEM	0x1000_0000	1 KB	Data RAM / stack
GPIO	0x2000_0000	16 B registers	LED / SW / KEY / HEX
Future	0x3000_0000+	Not implemented	Timer / UART / I2C / Accelerator

Important distinction

The table documents the **intended** map.

The actual decoder determines which addresses **physically respond**.

GPIO Register Map

Address	Register	Access	Meaning
0x2000_0000	LED	W	LEDG[9:0]
0x2000_0004	SW	R	SW[9:0]
0x2000_0008	KEY	R	KEY[2:0], assumed 1 = pressed
0x2000_000C	HEX	W	Four hexadecimal digits

Why offsets are 0, 4, 8, C

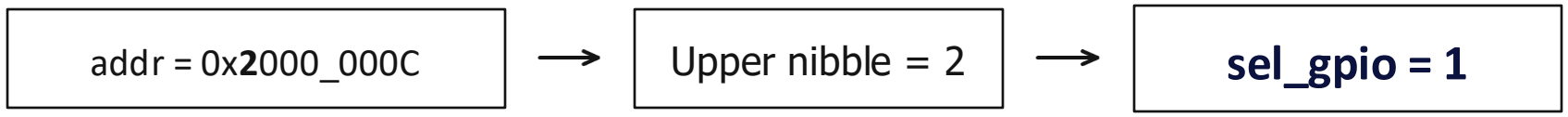
Each register is accessed as a 32-bit word → One word = 4 bytes → +0x00, +0x04, +0x08, +0x0C

Address Decoder

The CPU produces one address. The decoder decides which device owns it.

```
always @(*) begin
    sel_dmem = (addr[31:28] == 4'h1);
    sel_gpio = (addr[31:28] == 4'h2);
end
```

Example



Safe Write Enable

wdata can be connected to multiple slaves. But only the selected slave may change state.

```
assign we_dmem = we & sel_dmem;
assign we_gpio = we & sel_gpio;
```

Required property

If sel_gpio = 0:

GPIO must not write.

If the address is unmapped:

No peripheral state should change.

Read-Data MUX

Multiple slaves can produce read data. The CPU has only one rdata input.

```
always @(*) begin
    if      (sel_dmem) rdata = dmem_rdata;
    else if (sel_gpio) rdata = gpio_rdata;
    else               rdata = 32'hDEAD_BEEF;
end
```

DEAD_BEEF A recognizable debug value for an unmapped read.

Partial Address Decoding

Our simple decoder checks only addr[31:28], and GPIO internally checks addr[3:2].

32-bit address



Result

Different addresses may select the same physical register. This is called:

Address Aliasing

Aliasing Example

These addresses may all select the same LED register:

0x2000_0000
0x2000_0010
0x2ABC_0000

LED register

Why?

Upper nibble is 2 → GPIO selected.

addr[3:2] = 00 → LED register selected.

Other bits are ignored.

Partial decoding

Advantage: simple hardware

Cost: multiple addresses can refer to the same device

Full vs Partial Decoding

	Full Decoding	Partial Decoding
Compared bits	All required bits	Selected bits only
Hardware logic	More	Less
Address uniqueness	High	Lower
Aliasing	Avoided	Possible
Our lab	No	Yes

Engineering lesson

A memory map is not only a software document.
It is implemented by hardware comparison logic.

The Address Must Agree in Multiple Places

The same memory map appears in several layers.

Location	Example	Used By
SPEC.md	GPIO = 0x2000_0000	Human
Verilog	address decoder / register select	Hardware
link.ld	IMEM / DMEM placement	Linker
gpio.h	GPIO register addresses	C program

If these disagree, each part can be "correct" and the system still fails.

Hardware–Software Contract

Think of the memory map as an API.

Hardware promises

"If CPU accesses 0x2000_0004, I will return the switch value."

Software promises

"When I need the switch value, I will read 0x2000_0004."

The contract includes

Address	Read/write direction	Register width
Bit meaning	Reset behavior	Timing assumptions

Adding a New Peripheral

1 Allocate an unused address.	**2** Define its register map and bit meanings.	**3** Add sel_xxx in the address decoder.
4 Gate its write enable.	**5** Add its read data to the read-data MUX.	**6** Implement internal registers / state machine.
7 Connect external FPGA pins if required.	**8** Add matching definitions to software.	**9** Verify address boundaries and invalid accesses.

Same method works for GPIO, Timer, UART, I2C, or an accelerator.

03

Peripheral Hardware

Why this matters

Inputs must be synchronized, outputs must hold state, and every register behavior must be documented.

Input Peripheral: SW and KEY

An input peripheral connects external signals to the CPU.



Read path: peripheral → CPU

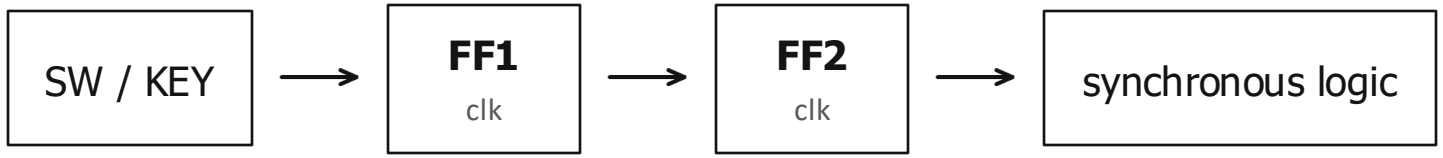
The CPU does not directly "see" a switch.
It sees a 32-bit value returned from an address.

Why Synchronization Is Required

SW and KEY are external to the CPU clock. Their transitions can occur near a clock edge. This can cause:

Metastability

A common protection technique is a 2-flip-flop synchronizer.



Synchronizer in Verilog

```
always @(posedge clk) begin
    sw_meta  <= SW;
    sw_sync  <= sw_meta;

    key_meta <= ~KEY;
    key_sync <= key_meta;
end
```

Why ~KEY?

The DE0 push-button input is active-low.

This design converts it to:

1 = pressed

before software reads it.

Synchronizer ≠ Debouncer

A 2-flip-flop synchronizer addresses:

Clock-domain metastability risk

It does **not** guarantee removal of:

Mechanical contact bounce

A push button may change 0/1 several times during one physical press.

Different problems require different solutions.

Output Peripheral: LED

An output peripheral needs state.

```
always @(posedge clk) begin
    if (rst)
        led_reg <= 10'b0;
    else if (we_gpio && addr[3:2] == 2'b00)
        led_reg <= wdata[9:0];
end

assign LEDG = led_reg;
```

Why a register? The CPU's store transaction lasts only briefly. The LED must hold its value after the CPU executes the next instruction.

Read-Only and Write-Only Registers

This GPIO design defines:

Read-only

SW · KEY

If software writes SW/KEY:

Hardware ignores the write.

Write-only

LED · HEX

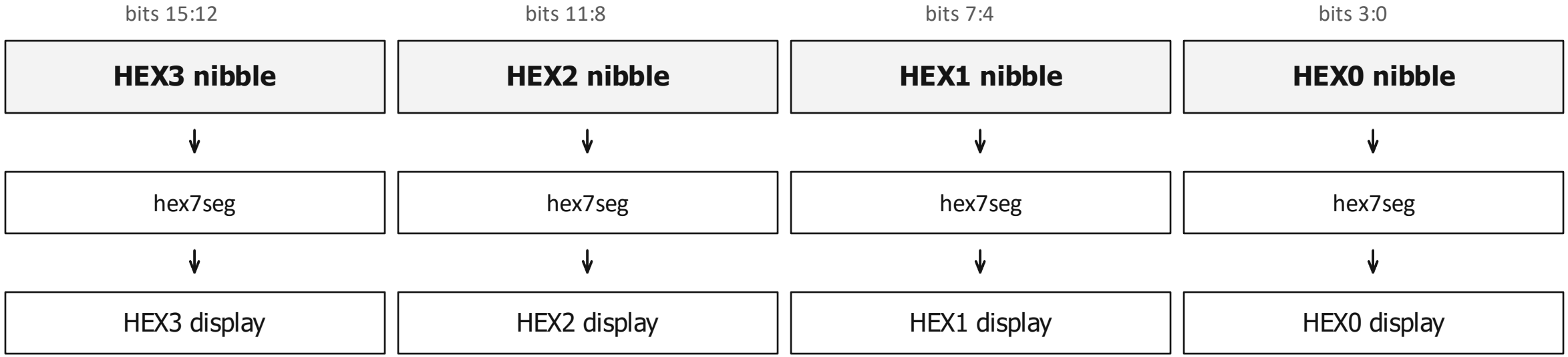
If software reads LED/HEX:

This design returns the default GPIO read value, not necessarily the output state.

The register map must document these behaviors.

7-Segment Output

HEX is a 16-bit logical display register.



Each nibble stores a hexadecimal digit: 0 … F

Active-Low 7-Segment

The display outputs are active-low:

0 = segment ON

Hardware decoder

The CPU writes a nibble.
Verilog converts it to physical segment signals.

Examples using {g,f,e,d,c,b,a}:

Digit	Pattern
0	1000000
1	1111001
2	0100100
8	0000000
F	0001110

Hardware Decoder vs Software Pattern

	Decode in Hardware	Pattern in Software
CPU writes	Digit 0–F	Segment pattern
Lookup table	Verilog	C / data memory
C code	Simple	More flexible
Display symbols	Limited	Flexible
System choice	Yes — used in this system	No

Why hardware decode is useful here

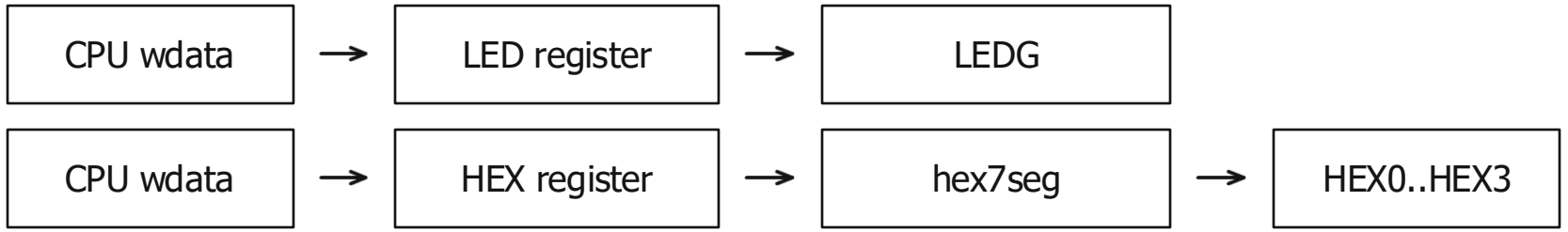
The CPU cannot read constant tables from IMEM through its data bus.

The Complete GPIO Data Path

Read · peripheral → CPU



Write · CPU → peripheral



GPIO is not "just pins". It is a memory-mapped hardware module.

04

C and Memory-Mapped I/O

Why this matters

C hides the bus. volatile pointers expose hardware registers, and the compiler may use instructions the custom core does not have.

Now Move to C

We want to write:

```
LED = 0x3FF;
```

instead of:

```
lui  t0, 0x20000
addi t1, x0, 1023
sw   t1, 0(t0)
```

C is an abstraction.

The compiler must eventually generate the machine instructions that our CPU can execute.

Hardware Register as a C Pointer

gpio.h

```
#define LED (*(volatile unsigned int *)0x20000000)
#define SW  (*(volatile unsigned int *)0x20000004)
#define KEY (*(volatile unsigned int *)0x20000008)
#define HEX (*(volatile unsigned int *)0x2000000C)
```

Then:

```
LED = 0x3FF;
unsigned int s = SW;
```

looks like ordinary variable access. **But it generates memory transactions.**

Read the Pointer from Inside Out

```
(*(volatile unsigned int *)0x20000004)
```

Step 1

0x20000004

A numerical address.

Step 2

(volatile unsigned int *)

Treat that number as a pointer to a 32-bit unsigned value.

Step 3

***(...)**

Access the value stored at that address.

Result

A C expression becomes memory-mapped I/O.

Why volatile Matters

Hardware can change without the C program assigning a value.

```
while (KEY == 0) {
}
```

KEY can change because a person presses a physical button.

Without volatile, an optimizing compiler may assume repeated reads are unnecessary.

volatile tells the compiler:

Perform the memory access as written.	Do not cache the value in a register indefinitely.	Do not remove a required hardware write as "unused".

What volatile Does NOT Mean

volatile is important, but it is not magic. It does **not**:

- Make an access atomic
- Synchronize clock domains
- Debounce a button
- Create a memory barrier for every architecture
- Fix an incorrect address
- Add unsupported CPU instructions

It controls compiler treatment of the access.

Hardware correctness still matters.

C Bit Operations for Hardware

Goal	C
Test bit n	x & (1u << n)
Set bits	x | mask
Toggle bits	x ^ mask
Select bits 4:0	x & 0x1F
Select bits 9:5	(x >> 5) & 0x1F

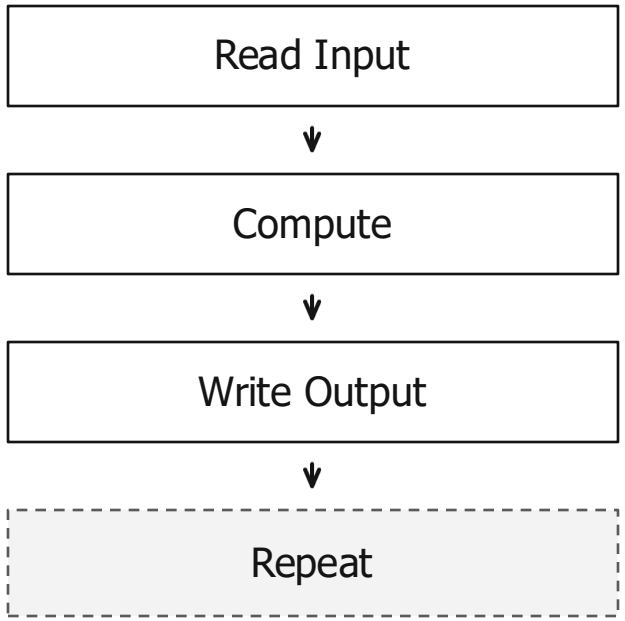
Example:

```
if (KEY & 0x2) {
    /* KEY[1] is pressed */
}
```

Bitwise operations connect naturally to hardware registers.

Bare-Metal Super Loop

There is no operating system in this lab. A simple embedded program repeatedly:



Example:

```
int main(void) {
    while (1) {
        LED = SW;
    }
}
```

This repeated checking is called:

Polling

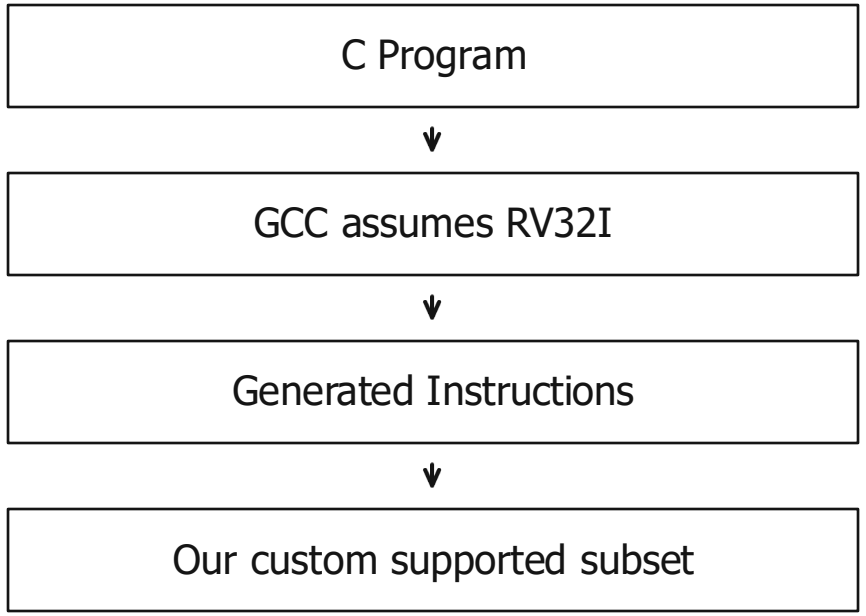
The Important Compiler Problem

We compile with:

```
-march=rv32i
```

But our educational CPU does **not necessarily implement every RV32I instruction**.

Therefore two contracts exist.



-march=rv32i is not enough to prove compatibility with this custom core.

Instruction Compatibility: Three Classes

Guaranteed

By the stated Chapter 10 subset

ADD SUB AND OR

XOR SLT SLL SRL

ADDI ANDI ORI

LW SW

BEQ BNE

LUI JAL JALR

Conditional

Verify the actual control.v

SRLI SLLI

XORI SLTI

These are RV32I instructions, but they are not in the stated guaranteed list.

Unsupported for this lab

Examples

BLT BLTU

BGE BGEU

Byte/halfword loads and stores if not implemented. Multiplication/division extensions.

Why C Can Surprise You

A simple C operator can generate an instruction you did not explicitly request.

C construct	Possible generated instruction
a < b	blt, bltu, slt, sltu
x >> 5	srli
x << 2	slli
~x	xori
char load	lb / lbu
short store	sh
a * b	multiply instruction or helper routine

Always inspect the generated code.

Safe C Style for This Core

Prefer

- unsigned int
- == and !=
- + and -
- &, |, ^
- Simple functions
- Explicit MMIO
- Local variables
- Small programs

Avoid unless verified

- <, >, <=, >=
- char / short memory operations
- *, /, %
- Library functions
- Initialized global data
- Constant lookup tables
- Compiler features that require unsupported instructions

C source is not the final truth.

The final truth is: the generated machine instructions.

Workflow



Never skip the disassembly/ISA check on this custom processor.

05

Bare-Metal Runtime and Linker

Why this matters

There is no operating system. Startup code and the linker script decide where the program starts, where data lives, and where the stack begins.

Why main() Is Not the First Instruction

A normal C environment has runtime startup code. Our bare-metal system does not.

Before main()

Stack pointer must be initialized.	Program entry must be defined.	Any required runtime initialization must be handled explicitly.

Therefore we provide:

crt0.S

Startup Code

```
.section .text.start
.globl _start

_start:
   lui  sp, %hi(_stack_top)
   addi sp, sp, %lo(_stack_top)

   jal  ra, main

hang:
   jal  x0, hang
```

Purpose

1. Set sp.
2. Call main.
3. If main returns, remain in a safe infinite loop.

Why Avoid la and call Here?

Pseudo-instructions may be expanded by the assembler/linker into instructions the custom core does not implement.

For example, position-independent or PC-relative sequences can involve:

AUIPC

Our stated core does not include AUIPC.

Course implementation rule

Use explicit startup sequences and inspect final disassembly.

> The exact expansion can depend on toolchain/linker behavior.

What Is the Stack?

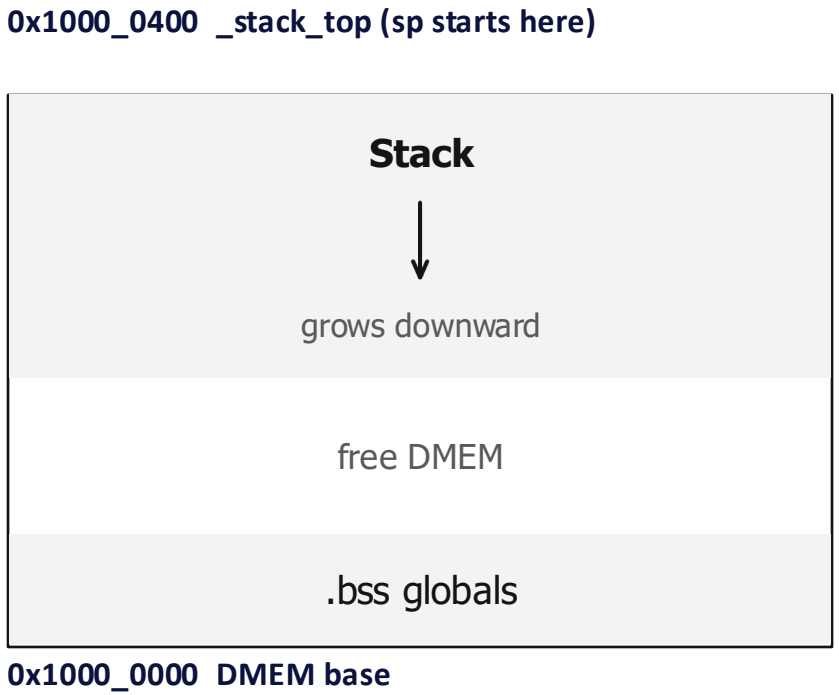
The stack stores temporary function state such as:

- Saved registers
- Return-related state
- Local variables when registers are insufficient

Our DMEM is 0x1000_0000 to 0x1000_03FF. Therefore:

_stack_top = 0x1000_0400

The stack grows downward into DMEM.



Linker Script = Software View of the Memory Map

```
MEMORY {
    IMEM (rx) : ORIGIN = 0x00000000, LENGTH = 1K
    DMEM (rw) : ORIGIN = 0x10000000, LENGTH = 1K
}
```

The linker needs to know:

Where instructions belong	Where writable data belongs	Where the stack starts

Hardware address map and linker address map must match.

Linker Sections

```
SECTIONS {
    .text : {
        *(.text.start)
        *(.text*)
    } > IMEM

    .rodata : { *(.rodata*) *(.srodata*) } > IMEM
    .data   : { *(.data*)   *(.sdata*)   } > DMEM
    .bss    : { *(.bss*) *(.sbss*) *(COMMON) } > DMEM
}
```

.text	.rodata	.data	.bss
Instructions	Read-only constants	Initialized writable globals	Zero-initialized globals

Why This Tiny SoC Restricts C Data

The CPU fetches instructions from IMEM. But:

The data bus cannot read IMEM.

Therefore a C constant stored in .rodata inside IMEM cannot be loaded as normal data.

Also, there is no complete runtime to copy initialized .data from a load image into DMEM.

This is a hardware/software architecture limitation, not a C syntax problem.

.bss Needs Special Care

In standard C execution, .bss variables are expected to begin as zero.

Our minimal crt0.S does not automatically clear .bss.

Therefore:

Do not rely on implicit zero initialization unless startup code explicitly clears .bss.

For this minimal system, prefer local variables initialized inside main(). This makes behavior explicit and avoids hidden runtime requirements.

Link-Time Guard Rails

The linker can reject unsupported program layouts.

```
ASSERT(SIZEOF(.rodata) == 0,
    "ERROR: const data in IMEM - CPU cannot read IMEM through data bus")

ASSERT(SIZEOF(.data) == 0,
    "ERROR: initialized global variable is not supported")
```

Better engineering

Detect incompatible software at build time instead of discovering it on the FPGA.

06

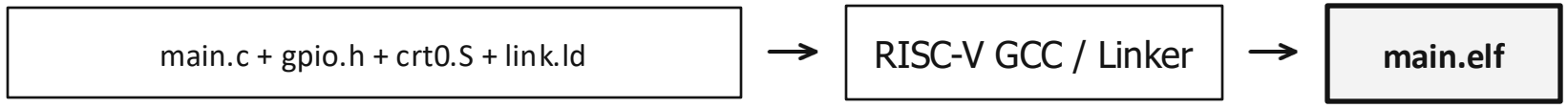
Compile / Disassemble / Verify

Why this matters

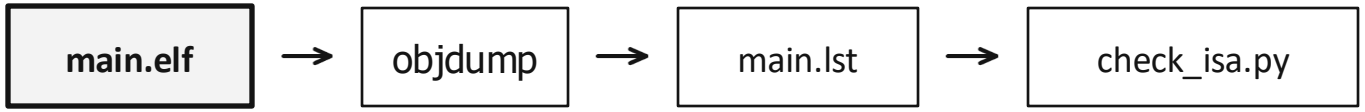
C source is not the final truth. The generated machine instructions must be inspected, checked, simulated, then deployed.

Complete Software Build Flow

1 · Compile and link



2 · Verify the instructions



3 · Build the memory image



Recommended Compile Command

```
riscv-none-elf-gcc \
  -march=rv32i \
  -mabi=ilp32 \
  -O0 \
  -ffreestanding \
  -nostdlib \
  -T link.ld \
  crt0.S main.c \
  -o main.elf
```

Toolchain prefix may differ

```
riscv-none-elf-
```

or

```
riscv64-unknown-elf-
```

The important part is the target architecture and generated instructions.

What the GCC Options Mean

Option	Meaning
-march=rv32i	Generate RV32I ISA code
-mabi=ilp32	32-bit int/long/pointer ABI
-O0	Easier C-to-assembly inspection
-ffreestanding	No hosted OS environment assumed
-nostdlib	Do not link the normal standard runtime/libraries
-T link.ld	Use our memory map

Reminder

-march=rv32i still allows RV32I instructions that our subset may not implement.

Disassemble Before Running

```
riscv-none-elf-objdump \
  -d -M no-aliases \
  main.elf > main.lst
```

Why no-aliases?

We want to see actual architectural instructions, instead of hiding them behind pseudo names such as:

j	ret	mv	li

For a custom CPU, exact instructions matter.

ISA Checker

Concept:

```
OK = {
    "add", "sub", "and", "or", "xor", "slt",
    "sll", "srl", "addi", "andi", "ori",
    "lw", "sw", "beq", "bne", "lui", "jal", "jalr"
}
```

For every disassembled instruction:

Supported?	**Conditional?**	**Unsupported?**
Continue	Warning + simulation	Build failure

The checker must match the actual RTL, not an assumed ISA.

Conditional Instructions Must Be Explicit

If the lab needs: `(s >> 5)` GCC may generate: **srli**

There are only two valid approaches.

Option A

Core supports SRLI

Document it in the supported-instruction list and verify it.

Option B

Core does not support SRLI

Do not use that C construct, or revise the lab.

Do not leave the CPU/software contract ambiguous.

Binary to $readmemh

After extracting the binary:

```
riscv-none-elf-objcopy \
  -O binary main.elf main.bin
```

convert every 4 little-endian bytes into one 32-bit hexadecimal word.

```
for (w,) in struct.iter_unpack("<I", data):
    f.write(f"{w:08x}\n")
```

→

Result

10000137
40010113
008000ef
…

One line = one 32-bit instruction word.

Load the Program into IMEM

```
initial begin
    $readmemh("main.hex", imem);
end
```

Simulation

Icarus Verilog loads main.hex.

FPGA

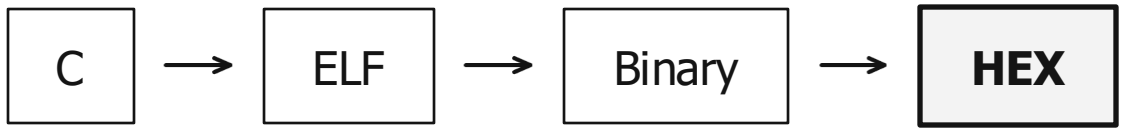
Quartus uses the initialized memory contents during compilation.

Consequence · after changing C

Rebuild the software image and rebuild the FPGA image when IMEM is synthesized into the design.

Two Build Processes

Software build



Hardware build



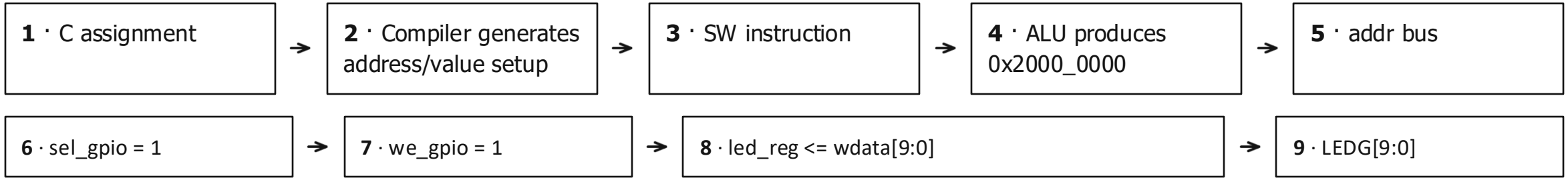
An important SoC concept

Software and hardware have different build flows but meet at the memory image and hardware/software interface.

Follow One C Statement End-to-End

```
LED = 0x3FF;
```

Possible conceptual sequence · write path: CPU → peripheral

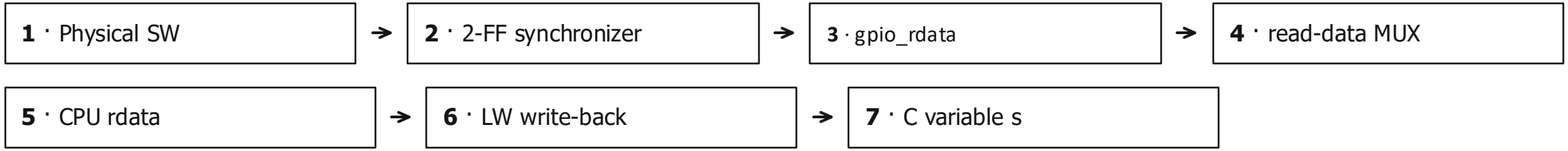


This is the bridge from software to physical hardware.

Follow a C Read End-to-End

```
unsigned int s = SW;
```

Hardware path · read path: peripheral → CPU



A C variable can contain the state of a physical FPGA pin.

Example Disassembly: What to Look For

For:

```
HEX = 0x1234;
```

expect a sequence conceptually similar to:

```
lui   a5, 0x20000
addi  a5, a5, 12
lui   a4, 0x1
addi  a4, a4, 0x234
sw    a4, 0(a5)
```

Observe

One C statement may require several instructions:

1. Construct peripheral address
2. Construct value
3. Store value

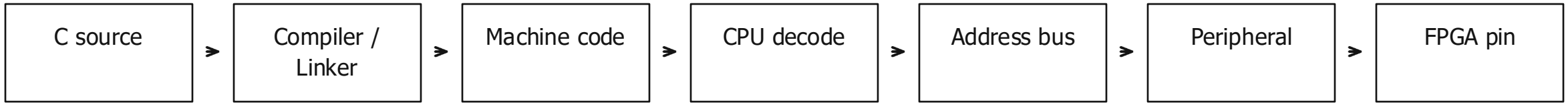
C hides details; the processor still executes instructions one by one.

Debugging: Start from the Symptom

Symptom	First Things to Check
Link error for helper routine	Unsupported C operation / missing library
.text does not fit	Program larger than IMEM
Nothing executes	_start, reset PC, main.hex
LED/HEX unchanged	MMIO address, sw, we_gpio
Switch value wrong	KEY/SW polarity, synchronizer, lw
Program jumps incorrectly	Unsupported instruction / branch / JALR
New C gives old FPGA behavior	HEX not regenerated or Quartus not rebuilt

Debugging Across Layers

A full-system bug can live in different layers.



Debug rule

Do not change Verilog immediately.

First identify the layer where expected behavior stops matching actual behavior.

07

Button Input and Counting

Why this matters

One physical press must produce one count. Polling, edge detection, and packed BCD make that work on this CPU.

Polling a Push Button: The First Problem

Incorrect idea:

```
if (KEY & 0x2) {
    count = count + 1;
}
```

If the button remains pressed for many CPU loop iterations:

Count increases many times.

We want:

One physical press → One count

Therefore software must detect the transition from: **not pressed → pressed**

Software Edge Detection

Store the previous state.

```
unsigned int changed = now ^ prev;

if (changed & now & 0x2) {
    /* KEY[1] rising edge */
}
```

Interpretation

changed Which bits changed?	**changed & now** Which changed bits are now 1?	**& 0x2** Was it KEY[1]?

This is a software version of an edge detector.

Synchronization vs Edge Detection vs Debouncing

These are three different jobs.

Problem	Solution
Asynchronous input / metastability	2-FF synchronizer
Count only once per transition	Edge detection
Mechanical contact produces repeated transitions	Debouncing

Do not use the terms interchangeably.

A synchronizer does not automatically debounce a switch.

Why BCD for the Counter?

HEX hardware displays each 4-bit nibble as one digit.

If binary count = 10, the register holds 0x000A and the display becomes:

000A

But we want decimal:

0010

Normally software could use /10 and %10. Our CPU does not provide the required arithmetic support for a normal division-based conversion.

Solution: store the count directly in Packed BCD.

Packed BCD

Example decimal 129: **0x0129**

HEX3	HEX2	HEX1	HEX0
0	1	2	9

Each nibble already represents one decimal digit.

Advantage

The same 16-bit value can be written directly to the HEX register. No division is needed.

Polling Is Simple, But Not Free

Polling repeatedly asks:

"Has something happened yet?"

Advantages

- Simple
- Easy to understand
- No interrupt hardware required

Limitations

- CPU spends time checking
- Event latency depends on polling rate
- Short events can be missed
- More peripherals make the loop more complex

This motivates interrupts in more advanced systems.

08

Full-System Extensions

Why this matters

Timer, UART, I2C, and accelerators extend the same address-bus model. The complexity is inside the peripheral.

Timer Peripheral

A timer is a counter controlled by registers. Possible register map:

Offset	Register	Purpose
+0x00	COUNT	Current count
+0x04	LIMIT	Compare value
+0x08	CTRL	Enable / reset
+0x0C	STATUS	Timeout flag

CPU view

Still only LW and SW.

The complexity is inside the peripheral.

UART Peripheral

UART communicates with a PC or another serial device.

Physical side

- TX
- RX
- Agreed baud rate
- Start bit
- Data bits
- Stop bit

CPU side · possible registers

- TXDATA
- RXDATA
- STATUS
- CONTROL

New software concept

The hardware operation takes time.

Software may need to wait until:

TX_READY = 1

I2C Peripheral

I2C uses

- SCL — clock
- SDA — data
- Multiple devices on the same bus

Two address spaces now exist

1. CPU address of the **I2C controller**
2. 7-bit I2C address of the **external sensor**



"Address" has meaning at different layers.

AI Accelerator as a Peripheral

A hardware accelerator can also be memory-mapped. Example control sequence:

1 CPU writes input data.	**2** CPU writes weights/configuration.	**3** CPU writes START = 1.
4 Accelerator computes.	**5** CPU polls DONE.	**6** CPU reads result.

Flexible CPU + Specialized Hardware

Same bus concept — very different internal hardware.

Peripheral Complexity Comparison

Peripheral	CPU Operation	Internal Complexity	Software Concern
LED	Write	Very low	Value
SW	Read	Very low	Input state
KEY	Read	Low	Edge / bounce
Timer	Read/write	Medium	Timing / status
UART	Read/write	Medium	Ready / busy
I2C	Sequence of accesses	High	Protocol
Accelerator	Command + data	High	Start / done / data

MMIO provides a common interface to very different hardware.

Full-System Debugging Example

Symptom: **HEX does not change.** Possible layers:

1	Is the C statement executed?	**6**	Is sel_gpio = 1?
2	Did GCC generate SW?	**7**	Is we_gpio = 1?
3	Is the target address 0x2000_000C?	**8**	Does hex_reg change?
4	Does CPU decode SW correctly?	**9**	Does hex7seg decode correctly?
5	Is addr correct?	**10**	Are FPGA pins assigned correctly?

Debugging becomes systematic when interfaces are explicit.

AI-Assisted Development Workflow

SPEC → **GENERATE** → **INSPECT** → **SIMULATE** → **EXPLAIN**

AI can help

- Draft C code
- Explain disassembly
- Draft linker scripts
- Find address mismatches
- Generate test ideas

AI must not be trusted to decide silently

- Which instructions the custom CPU supports
- Whether an address map matches RTL
- Whether a compiler-generated sequence is executable
- Whether an FPGA input is synchronized correctly

Verification remains an engineering responsibility.

AI Failure Challenge

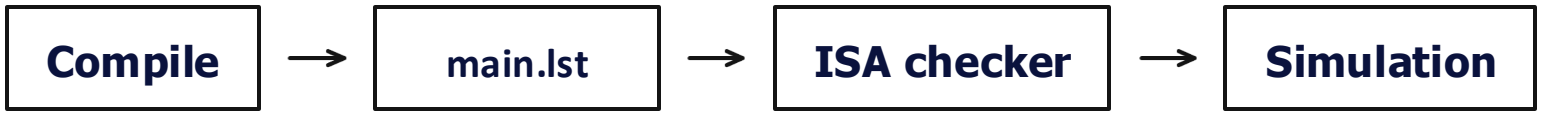
An AI assistant suggests:

```
for (int i = 0; i < 100; i++) {
    LED = i;
}
```

Questions

1. Which comparison instructions might GCC generate?
2. Does the custom core support them?
3. What happens if it does not?
4. How would you discover the problem before FPGA programming?

Correct process



Common Mistakes

Hardware–software mismatch

gpio.h address ≠ Verilog address

Unsupported generated instruction

C looks valid but CPU cannot execute the machine code

Wrong KEY polarity

Count occurs on release instead of press

Missing volatile

Compiler removes/reuses MMIO accesses

Wrong memory assumptions

Constant/global data placed where CPU cannot read it

Old program image

C changed but main.hex or FPGA image was not rebuilt

09

Review

Why this matters

A full system works only when the ISA, memory-map, timing, and build contracts all agree.

The Four Contracts of This SoC

1

ISA Contract

Compiler ↔ CPU

2

Memory-Map Contract

Software ↔ Address Decoder / Peripheral

3

Timing Contract

External input ↔ Clocked hardware

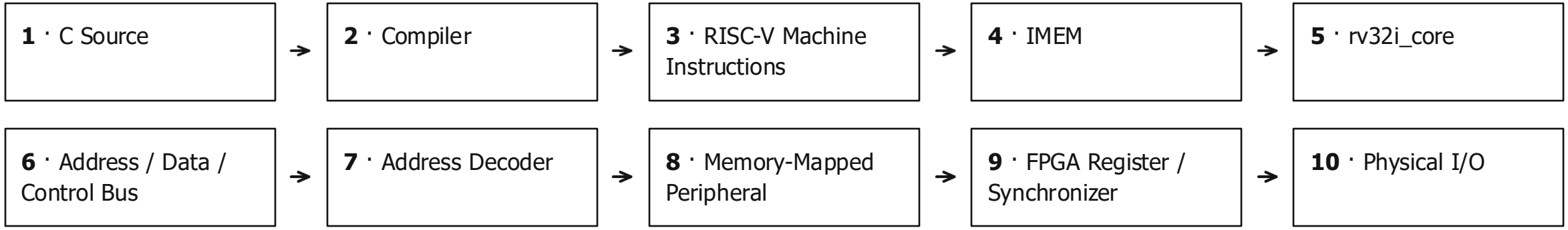
4

Build Contract

C / linker / HEX ↔ FPGA design

A full system works only when all contracts agree.

The Complete Story



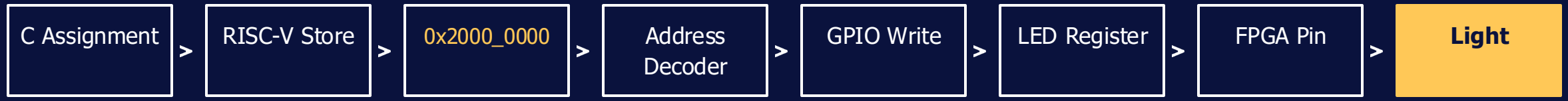
This is a complete hardware–software system.

Key Takeaways

- SoC = CPU + Memory + Interconnect + Peripherals.
- Memory-mapped I/O lets CPU use ordinary loads/stores for devices.
- The memory map is a hardware–software contract.
- Input synchronization and output registers solve different hardware problems.
- volatile makes MMIO accesses visible to the compiler as real side effects.
- Bare-metal C requires startup and linker control.
- A custom RV32I subset requires inspection of generated instructions.
- C code must be compiled, disassembled, checked, simulated, then deployed.
- UART, I2C, Timer, and accelerators extend the same address-bus model.

From C to a Physical LED

LED = 0x3FF; becomes:



Software works because hardware gives every operation a precise meaning.

A

Appendix

Reference files

The GPIO header, startup code, linker script, and helper scripts used in the build flow.

[Release xPack GNU RISC-V Embedded GCC v15.2.0-1 · xpack-dev-tools/riscv-none-elf-gcc-xpack · GitHub]